

Deployment Guide

Configuration, local Docker, Railway and how to start a new project from the template.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	Deployment
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

Configuration (env vars)

Local

Railway

New project from the template

Files in the repository

Introduction

Status: **draft v0.1 for review**. Items marked *verify* are assumptions to check before relying on them.

Configuration (env vars)

Var	Meaning
<code>DATABASE_URL</code> / <code>POSTGRES_*</code>	DB connection (API and migrations)
<code>CITYDB_SRID</code> , <code>CITYDB_HEIGHT_EPSG</code>	Fixed at the first start of the db volume (e.g. 32750 / 4979)
<code>SECRET_KEY</code> , <code>DEBUG</code> , <code>ALLOWED_HOSTS</code>	Django
<code>ALLOWED_ORIGINS</code>	CORS allow-list for the frontend
<code>PROJECT</code>	Seed folder name under <code>db/seeds/</code>

`.env.example` is committed; `.env` is git-ignored.

Local

```
cp .env.example .env , then make up (db + api) → make migrate → make seed → open  
http://localhost:8000/admin , /viewer/ , /api/v1/docs . make test runs migration + invariant tests.  
Pinned image: 3dcitydb/3dcitydb-pg:16-3.4-5.1.4 ; citydb-tool 1.4.0.
```

Railway

Services (same Dockerfiles as local, `railway.toml` per service): 1. **db** - image `3dcitydb/3dcitydb-pg:16-3.4-5.1.4`, volume mounted at the Postgres data dir, private networking only, env `SRID`, `HEIGHT_EPSG`, `POSTGRES_PASSWORD`. *verify*: init works on a Railway volume (may need `PGDATA` subdirectory), volume size/plan cost for the LOD1 data, image runs on Railway's amd64 hosts. 2. **api** - Django; pre-deploy command `dbmate up && psql -f db/seeds/$PROJECT/*.sql`; healthcheck `/api/v1/health`; public domain; `DATABASE_URL` from the private network. 3. **backup** (cron) - nightly `pg_dump -Fc` to an S3-compatible bucket. Do not rely on volume snapshots alone. 4. `citydb-tool imports` run as a one-off container (local or Railway) over the private network or a temporary TCP proxy; close the proxy afterwards.

Steps: create project → add db service (volume) → wait healthy → add api service from the GitHub repo → set env → deploy → create first admin user (`python manage.py createsuperuser` via Railway shell) → run `newman` smoke test against the Railway environment. Rollback: redeploy the previous api deployment; DB changes are forward-only migrations (use `dbmate down` only for the latest, before data depends on it); data: restore from dump.

New project from the template

Copy/fork the repo → change `.env` (SRID, project name) → add `db/seeds/<project>/` → `make up migrate seed` . Add themes by inserting into `ref_theme` (admin UI).

Files in the repository

File	Purpose
<code>docker-compose.yml</code>	Local db and api
<code>api/Dockerfile</code> , <code>api/entrypoint.sh</code>	api image; on start: dbmate up, create login roles, seeds, Django migrate, groups and first admin, gunicorn
<code>railway.toml</code>	api service on Railway (Dockerfile build, health check <code>/api/v1/health</code>)
<code>deploy/db/Dockerfile</code> , <code>deploy/db/railway.toml</code>	db service with <code>PGDATA</code> in a subfolder of the volume
<code>deploy/backup/</code>	nightly dump job (cron <code>0 18 * * *</code>), tested locally for dump and restore

Database logins: `twin_app` (Django admin; Django groups decide who may edit or delete) and `twin_api` (read-only API). Passwords come from `APP_DB_PASSWORD` and `API_DB_PASSWORD` . `ADMIN_EDIT_ENABLED=0` keeps the admin read-only (checkpoint); set `1` to enable editing.